Relatório técnico — Acervo

Projeto de intervenção em Refatoração e Orientação a Objetos — Situação-Problema 3.

Nome: _____

RGM: _____

1. Introdução, contexto e justificativa

O Acervo é um catálogo pessoal de livros, filmes e séries desenvolvido como solução para a Situação-Problema 3 do material Projetos Computacionais: da Teoria à Prática, elaborado por Artur Marques. O enunciado, nas páginas 39–40 do PDF fornecido, descreve uma aplicação cujo CRUD e busca foram implementados diretamente em uma Servlet. O resultado é uma concentração de responsabilidades que dificulta a manutenção e duplica o acesso ao banco.

O objetivo da intervenção é preservar o comportamento funcional exigido e distribuir as responsabilidades em classes coesas. A interface web permite cadastrar, listar, consultar detalhes, editar, excluir e pesquisar obras. O armazenamento utiliza H2 em arquivo por meio de JDBC. As tecnologias centrais permanecem alinhadas ao material: Java, POO, Servlets, JSP, JDBC e um banco relacional.

Não foi fornecido um repositório legado. Por isso, o cenário inicial é representado por um exemplo didático, explicitamente identificado como tal. A entrega é uma implementação completa da arquitetura proposta, e não uma alegação de alteração de um código original que o documento não disponibiliza. A refatoração é explicada como processo aplicável ao cenário, e a preservação funcional é demonstrada por requisitos e testes da implementação resultante.

2. Objetivos e critérios de sucesso

O objetivo geral é melhorar a organização, reutilização e testabilidade do catálogo. A solução deve satisfazer três critérios: nenhuma instrução SQL na Servlet ou no serviço; regras de validação centralizadas no serviço; e possibilidade de testar essas regras com uma implementação DAO substituta, sem iniciar HTTP ou JDBC.

Também se exige que todas as operações mandatórias continuem disponíveis pela interface web, que os dados sejam conservados no banco em arquivo e que falhas previsíveis produzam respostas compreensíveis, sem exibir SQL, credenciais ou rastros internos na página.

2.1 Requisitos funcionais e rastreabilidade

ID	Requisito	Implementação	Evidência principal
RF01	Interface web	JSP, CSS e JavaScript	Testes HTTP e capturas de tela
RF02	Cadastrar obra	POST /itens/salvar	fluxoCompletoCadastroBuscaEdicaoExclusao
RF03	Listar itens	GET /itens	crudRealComChaveGerada e teste HTTP
RF04	Visualizar detalhes	GET /itens/detalhes	fluxoCompletoCadastroBuscaEdicaoExclusao
RF05	Editar obra	GET /itens/editar e POST /itens/salvar	atualizaMantendoIdentificador e teste HTTP
RF06	Excluir obra	POST /itens/excluir	excluiItemExistente e teste HTTP
RF07	Buscar título ou autor/diretor	GET /itens?q=termo	buscaPorTituloOuAutorSemDiferenciarMaiusculas
RF08	Persistir em banco	JdbcItemMidiaDao e H2	dadosSobrevivemAoFechamentoEReabertura
RF09	Validar campos	ItemMidiaService	testes parametrizados de limites
RF10	Apresentar falhas sem quebrar o fluxo	Exceções e JSP de erro	testes HTTP de 400, 403, 404, 422 e 503

2.2 Requisitos não funcionais e limites

ID	Requisito	Decisão verificável
RNF01	Separação de responsabilidades	Pacotes model, dao, service, web e config
RNF02	Reutilização	Métodos preencher, mapear, consultar e modificar no DAO
RNF03	Testabilidade	Serviço depende de ItemMidiaDao e de Clock injetáveis
RNF04	Segurança básica	SQL parametrizado, escape HTML, validação e CSRF
RNF05	Usabilidade	Rótulos, foco visível, mensagens por campo e confirmação de exclusão
RNF06	Instalação reproduzível	pom.xml com versões fixadas e inicializador PowerShell
RNF07	Preservação de dados	CREATE TABLE IF NOT EXISTS e H2 em arquivo

Não foi definido nem medido um SLA de desempenho. A listagem sem paginação e a busca com LIKE são adequadas para uma coleção acadêmica pequena. Não se afirma conformidade integral com normas de acessibilidade ou segurança; foram aplicadas práticas concretas e descritas neste relatório.

3. Metodologia e planejamento

A Aprendizagem Baseada em Projetos orientou a passagem do problema à solução verificável. O desenvolvimento foi organizado em iterações: requisitos e modelos; domínio e persistência; validação e controle HTTP; interface e refinamento; testes, documentação e apresentação. Cada iteração produz um resultado que pode ser inspecionado.

Etapa	Entrega	Estimativa
Análise	Escopo, requisitos e critérios	3 horas
Modelagem	Casos de uso, classes, DER e sequência	3 horas
Implementação	Catálogo completo em camadas	8 horas
Testes e refinamento	Suítes e correções	4 horas
Documentação	Relatório, guia e manual	4 horas
Revisão e apresentação	Conferência e demonstração	2 horas
Total planejado	Projeto individual	24 horas

As horas são uma estimativa pedagógica, não o tempo medido do estudante ou da execução automatizada. O estudante deve registrar seu próprio esforço antes da entrega.

4. Tecnologias e decisões

Java 17 foi escolhido como versão-base compatível com o ambiente verificado. O Tomcat 10.1 fornece o ambiente Servlet 6.0 e JSP 3.1. As páginas utilizam Jakarta Tags/JSTL para iteração e escape de saída. JDBC mantém explícitas as consultas e o gerenciamento dos recursos, favorecendo a compreensão do papel do DAO. O H2 é um SGBD relacional Java que permite executar o projeto com persistência em arquivo sem instalar um servidor de banco separado. Documentação do Tomcat, Jakarta Tags, H2.

O Maven organiza dependências, compilação, testes e geração do WAR; o JUnit executa os testes de serviço, persistência e integração HTTP. As versões exatas constam de pom.xml. O inicializador local utiliza Tomcat embutido, mas a aplicação também é empacotada como WAR para Tomcat externo. Apache Maven, JUnit 5.

Não foram adicionados Spring, ORM ou uma API REST paralela, pois Servlet, JSP e JDBC tornam visível a divisão de responsabilidades estudada. A entidade única com enum evita subclasses sem comportamento distinto. A interface DAO fornece o ponto de substituição relevante ao problema.

5. Modelagem

5.1 Casos de uso

O ator é o usuário do catálogo pessoal. Ele pode cadastrar, listar, buscar, visualizar detalhes, editar e excluir obras. Editar e excluir operam sobre um identificador existente; a exclusão exige confirmação na interface. Todas as escritas passam por validação ou verificação apropriada ao tipo de operação.

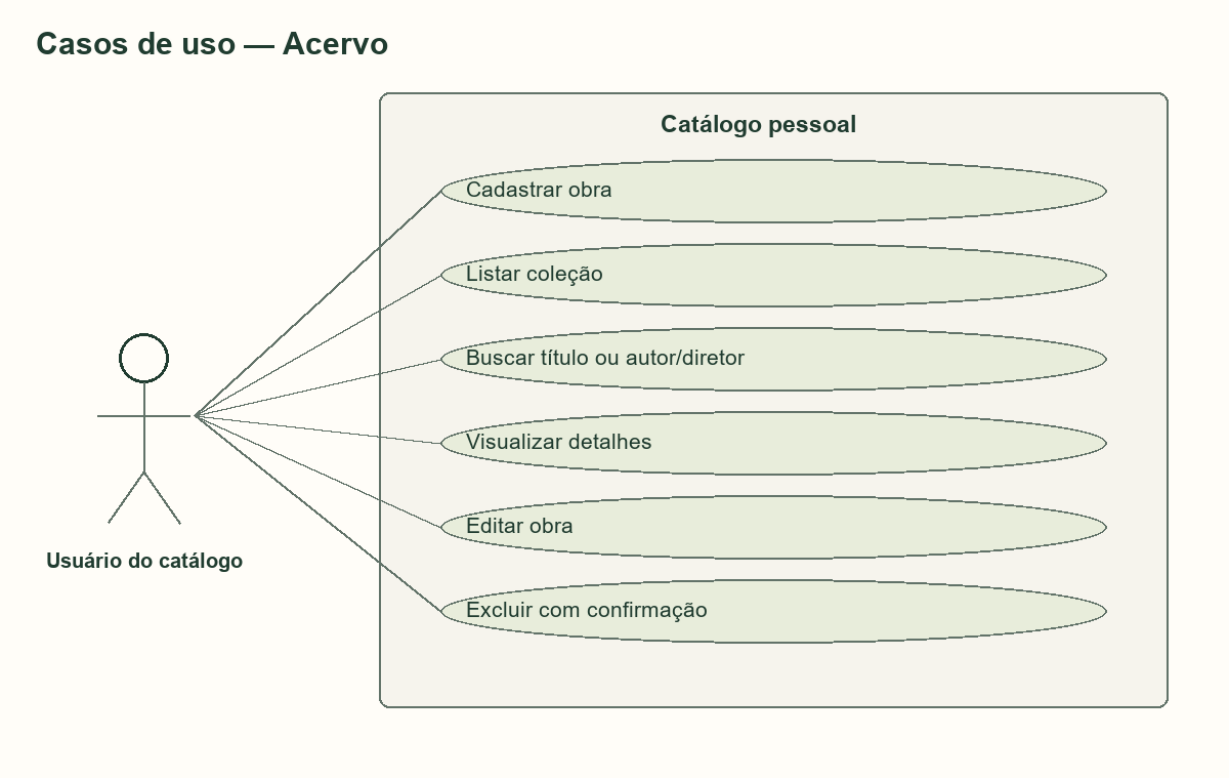


Diagrama de casos de uso

Os fontes editáveis ficam em docs/diagramas. O conjunto de casos de uso delimita o escopo funcional sem introduzir login, empréstimos ou upload.

5.2 Classes e dependências

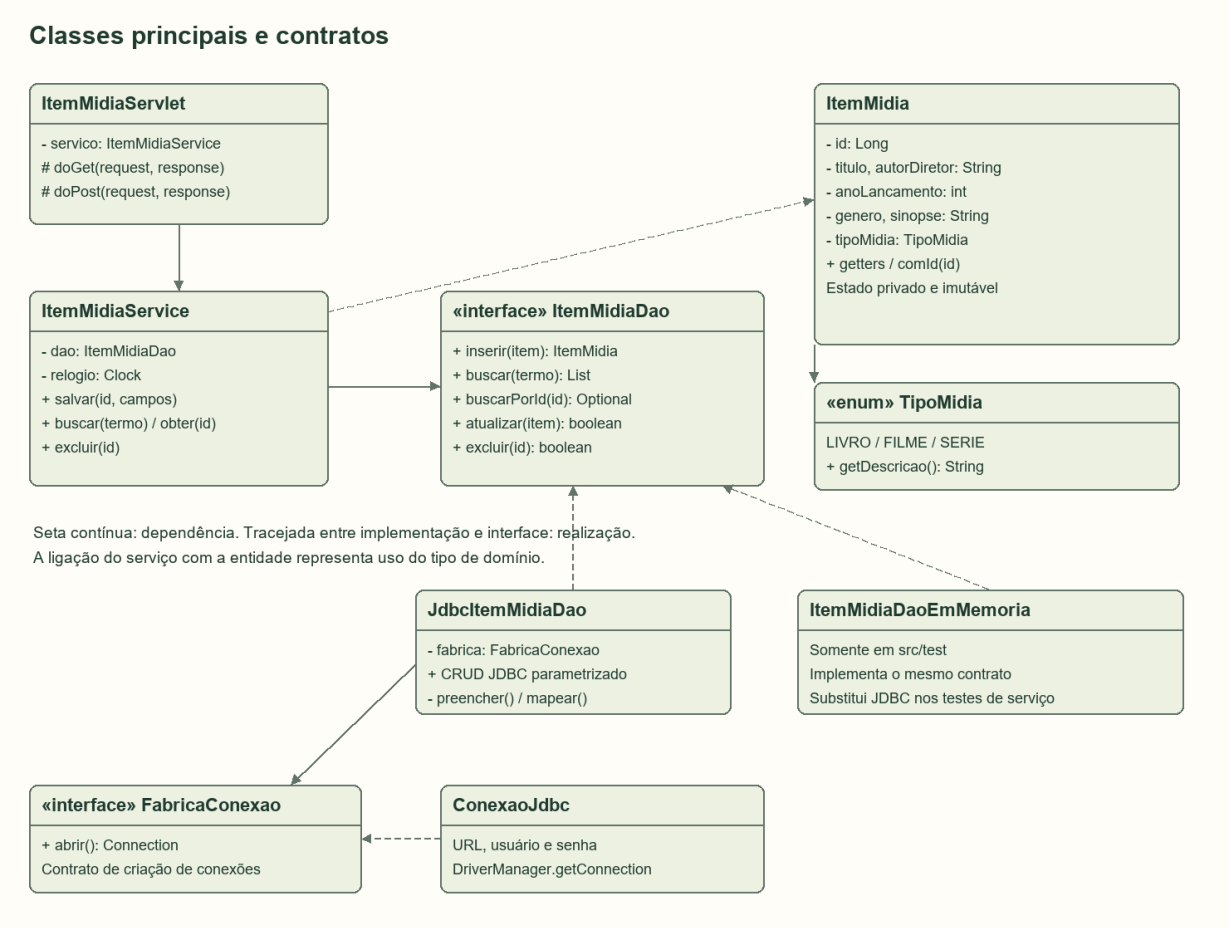


Diagrama principal de classes

ItemMidia encapsula os atributos da obra. TipoMidia representa as categorias válidas. ItemMidiaService recebe a interface ItemMidiaDao pelo construtor. JdbcItemMidiaDao implementa esse contrato e recebe FabricaConexao. ConexaoJdbc implementa a fábrica com DriverManager. ItemMidiaServlet conhece o serviço, mas não abre conexões. AplicacaoListener monta o conjunto de objetos na inicialização.

O DAO em memória existe somente em src/test. Ele implementa o mesmo contrato JDBC visto pelo serviço, possibilitando testes sem infraestrutura. Essa troca é um exemplo de polimorfismo por interface. Não há dependência da camada DAO em relação à Servlet ou às JSPs.

5.3 Modelo de dados

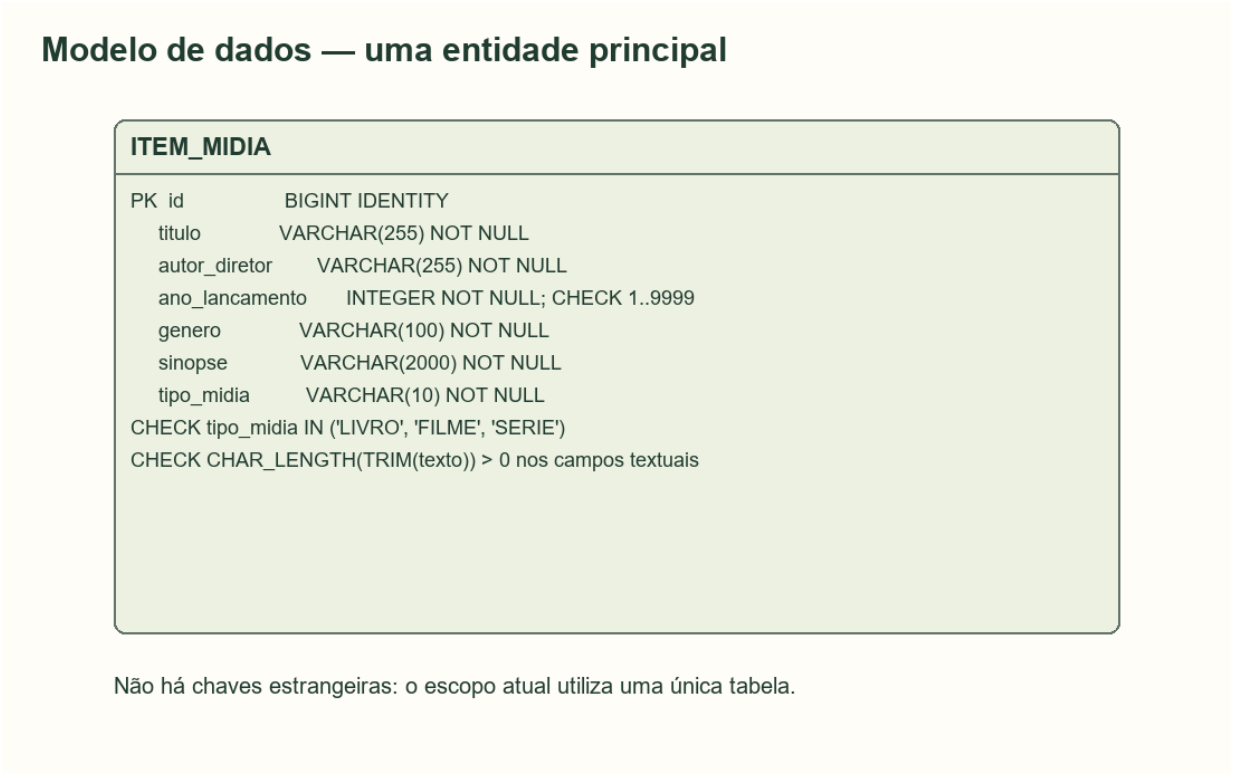


Diagrama entidade-relacionamento

Coluna	Tipo SQL	Restrição e significado
id	BIGINT	Chave primária gerada pelo banco
titulo	VARCHAR(255)	Obrigatório e não vazio
autor_diretor	VARCHAR(255)	Obrigatório e não vazio
ano_lancamento	INTEGER	Entre 1 e 9999 no banco
genero	VARCHAR(100)	Obrigatório e não vazio
sinopse	VARCHAR(2000)	Obrigatória e não vazia
tipo_midia	VARCHAR(10)	LIVRO, FILME ou SERIE

O arquivo executável src/main/resources/db/schema.sql é a fonte única do esquema. Não há relacionamentos entre tabelas porque há apenas uma entidade principal. Obras diferentes podem compartilhar o mesmo título; não foi imposta uma unicidade não solicitada. O serviço aplica a regra de ano até o próximo ano do calendário, enquanto o banco mantém o limite estrutural estável de 1 a 9999.

6. Arquitetura e processo de refatoração

6.1 Situação anterior, representada didaticamente

```
// Exemplo conceitual: não é código extraído de uma aplicação legada.
protected void doPost(HttpServletRequest req, HttpServletResponse res) {
    // 1. Lê parâmetros HTTP.
    // 2. Converte e valida os campos.
    // 3. Abre conexão e escolhe INSERT ou UPDATE.
    // 4. Preenche parâmetros, executa SQL e trata SQLException.
    // 5. Repete partes desse processo para outras operações.
    // 6. Monta a resposta ou redireciona o usuário.
}
```

A falha de organização não é o tamanho isolado de um método, mas a presença de motivos de mudança diferentes no mesmo componente. Uma alteração de SQL, uma regra de ano e uma nova navegação obrigariam a modificar a mesma Servlet. Repetir a tradução de ResultSet em objetos e a configuração das conexões multiplica os lugares sujeitos a inconsistência.

6.2 Transformação proposta

- Identificar os comportamentos que precisam permanecer: CRUD e busca.
- Extrair o estado da obra para ItemMidia e os tipos aceitos para TipoMidia.
- Definir o contrato ItemMidiaDao, sem dependência de HTTP ou JDBC.
- Mover SQL, parâmetros e conversão dos registros para JdbcItemMidiaDao.
- Centralizar conexão e credenciais em ConexaoJdbc e reutilizar a fábrica.
- Mover validação e coordenação das operações para ItemMidiaService.
- Fazer a Servlet traduzir entradas e resultados entre HTTP e serviço.
- Manter a apresentação nas JSPs e recursos estáticos.
- Verificar os comportamentos por testes nas três camadas relevantes.

Aspecto	Cenário do enunciado	Implementação entregue
SQL	Misturado ao controlador	JdbcItemMidiaDao
Validação de negócio	Acoplada ao HTTP	ItemMidiaService
Conexões	Repetidas por operação	FabricaConexao e try-with-resources
Conversão de registros	Repetida em consultas	mapear(ResultSet)
Parâmetros de escrita	Repetidos em INSERT/UPDATE	preencher(PreparedStatement, ItemMidia)
Teste de regras	Exige infraestrutura acoplada	DAO em memória e Clock fixo
Apresentação	Potencialmente misturada ao fluxo	JSP/JSTL em WEB-INF

Essa comparação é estrutural. Não foram calculados percentuais de redução de linhas ou de velocidade de desenvolvimento, pois não há um código legado executável nem medição comparativa.

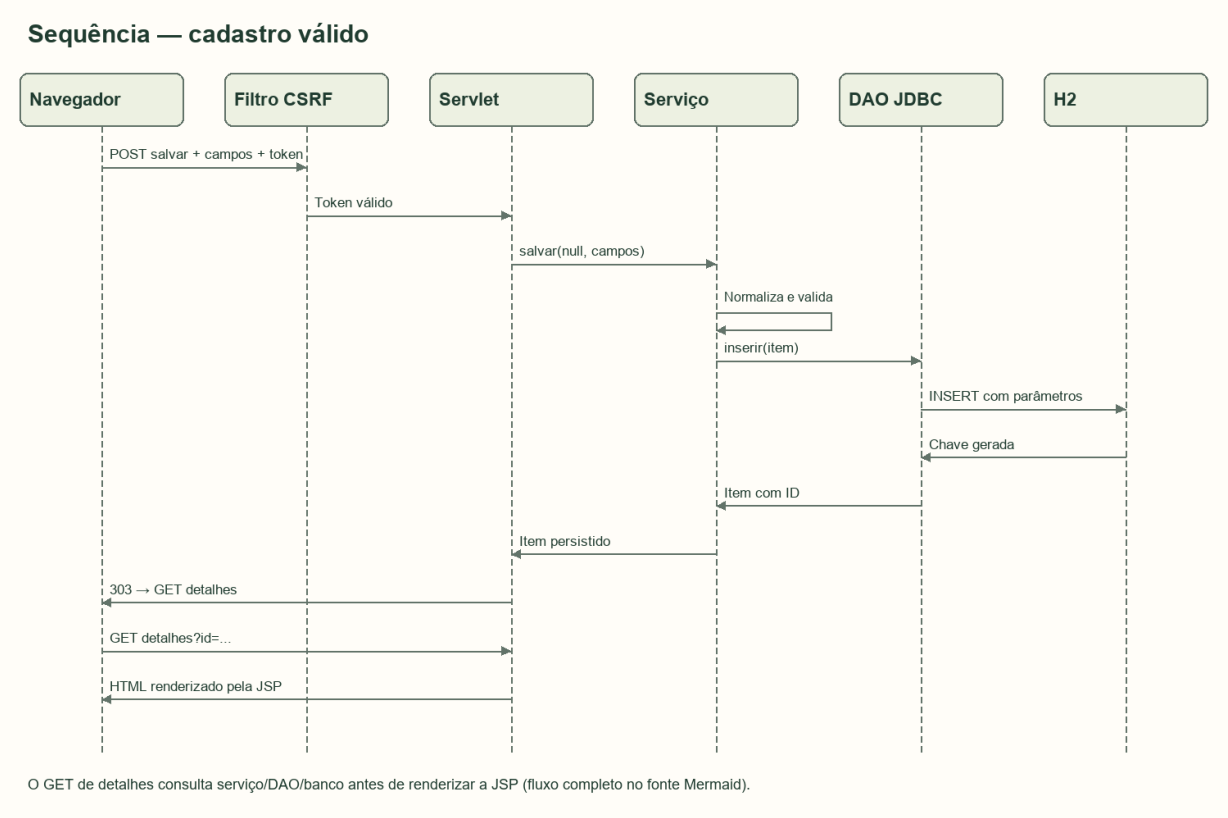
6.3 POO aplicada

O encapsulamento aparece nos atributos privados e finais da entidade e nas dependências privadas das classes. A abstração aparece nos contratos de persistência e de criação de conexão. O polimorfismo permite que ItemMidiaService utilize tanto JdbcItemMidiaDao quanto o DAO em memória. A composição constrói o serviço com o DAO e o DAO com a fábrica, em vez de criar essas dependências repetidamente em cada operação.

A herança é usada naturalmente quando ItemMidiaServlet estende HttpServlet, sobrescrevendo doGet e doPost. Não foram criadas subclasses de mídia artificiais: a existência de um tipo diferente não implica, por si só, comportamento diferente. Se regras específicas surgirem, essa decisão pode ser revista com base em requisitos concretos.

7. Funcionalidades-chave

7.1 Cadastro e edição



Sequência de cadastro

O usuário abre GET /itens/novo. A Servlet prepara os tipos disponíveis e encaminha para formulario.jsp. O filtro cria ou reutiliza um token de sessão inserido em campo oculto. No POST /itens/salvar, o filtro confere o token; a Servlet lê os campos; o serviço normaliza e valida; e o DAO executa o INSERT. O banco gera o ID. A Servlet retorna 303 com o endereço da página de

detalhes, seguindo Post/Redirect/Get.

```
ItemMidia item = servico.salvar(identificador, campos);
// O controlador não conhece a instrução INSERT ou UPDATE.
```

Na edição, o formulário traz um ID e os valores atuais. O mesmo método salvar valida os mesmos campos e chama atualizar. Se nenhuma linha for alterada, o serviço lança NaoEncontradoException, sem criar um novo registro. Em caso de erro de validação, o controlador responde 422, reapresenta os valores enviados e inclui mensagens específicas por campo.

7.2 Busca

A busca recebe um termo de até 255 caracteres, remove espaços nas extremidades e pesquisa título OU autor/diretor. O DAO usa LOWER para comparação sem diferença de maiúsculas e minúsculas. Um termo vazio lista tudo. A ordenação é por título em minúsculas e ID como desempate. Acentos continuam significativos.

```
SELECT id, titulo, autor_diretor, ano_lancamento, genero, sinopse, tipo_midia
FROM item_midia
WHERE LOWER(titulo) LIKE ? ESCAPE '!'
   OR LOWER(autor_diretor) LIKE ? ESCAPE '!'
ORDER BY LOWER(titulo), id
```

O padrão de pesquisa é passado por setString. O código escapa !, % e _ para procurar esses caracteres literalmente. A concatenação dos sinais % com o termo produz apenas um valor de parâmetro, não uma instrução SQL. Nenhuma entrada do usuário é concatenada ao comando SQL.

7.3 Exclusão e tratamento de falhas

A exclusão exige POST e token válido, e sua confirmação é apresentada nos detalhes. GET /itens/excluir não apaga dados e retorna 405. IDs ausentes, negativos, não numéricos ou fora do intervalo de long retornam 400. Um ID válido inexistente retorna 404. Falhas de persistência durante as operações retornam 503 com uma mensagem amigável e registram a causa no log do servidor.

O DAO transforma SQLException em PersistenciaException. O serviço não conhece o mecanismo JDBC. O controlador define a tradução HTTP dessa falha. A responsabilidade de apresentação não é colocada no DAO.

8. Segurança e integridade

Todas as consultas com dados variáveis utilizam PreparedStatement. O único Statement simples executa o esquema SQL estático, empacotado no projeto, durante a inicialização. try-with-resources fecha ResultSet, Statement e Connection inclusive quando ocorre erro. Como cada escrita atual consiste em uma única instrução, utiliza-se o autocommit JDBC; operações futuras com várias escritas relacionadas exigirão transações explícitas.

Os textos apresentados usam c:out ou fn:escapeXml. Valores numéricos gerados pelo banco e constantes do enum são exibidos diretamente. O filtro define UTF-8, X-Content-Type-Options, X-Frame-Options, Referrer-Policy e Content-Security-Policy. Os arquivos JavaScript e CSS são locais; não há necessidade de scripts inline ou de bibliotecas externas no navegador.

As operações POST exigem token CSRF aleatório de 256 bits associado à sessão. O cookie da sessão é HttpOnly e a aplicação usa rastreamento por cookie. O token não é autenticação nem restringe o acesso a usuários cadastrados. O servidor local escuta em 127.0.0.1 e não expõe um console web H2. Não há upload de arquivos nem execução de SQL digitado no navegador.

A configuração padrão sem senha é destinada ao banco embutido da demonstração local. As credenciais podem ser definidas por variáveis de ambiente. Uma implantação pública exigiria autenticação, autorização, HTTPS, política de cookies adequada, limitação de requisições, monitoramento e revisão de operação. Esses recursos não são declarados como implementados.

9. Estratégia de testes e evidências

ItemMidiaServiceTest utiliza DAO em memória e relógio fixo para verificar regras e fronteiras. JdbcItemMidiaDaoTest usa H2 real isolado para verificar CRUD, busca, restrições e armazenamento em arquivo. CatalogoHttpIT inicia Tomcat numa porta livre e executa requisições reais com cookies; assim, compila e exercita as JSPs e as demais camadas.

Os testes de serviço e JDBC são executados pelo Surefire na fase test. Os testes HTTP são executados pelo Failsafe após a geração do WAR na fase integration-test. O comando de conferência completo é `maven.ps1 verify`. Apenas `mvn test` não executa a suíte HTTP. O teste de arquivo utiliza uma pasta temporária exclusiva, sem tocar em data/catalogo.mv.db.

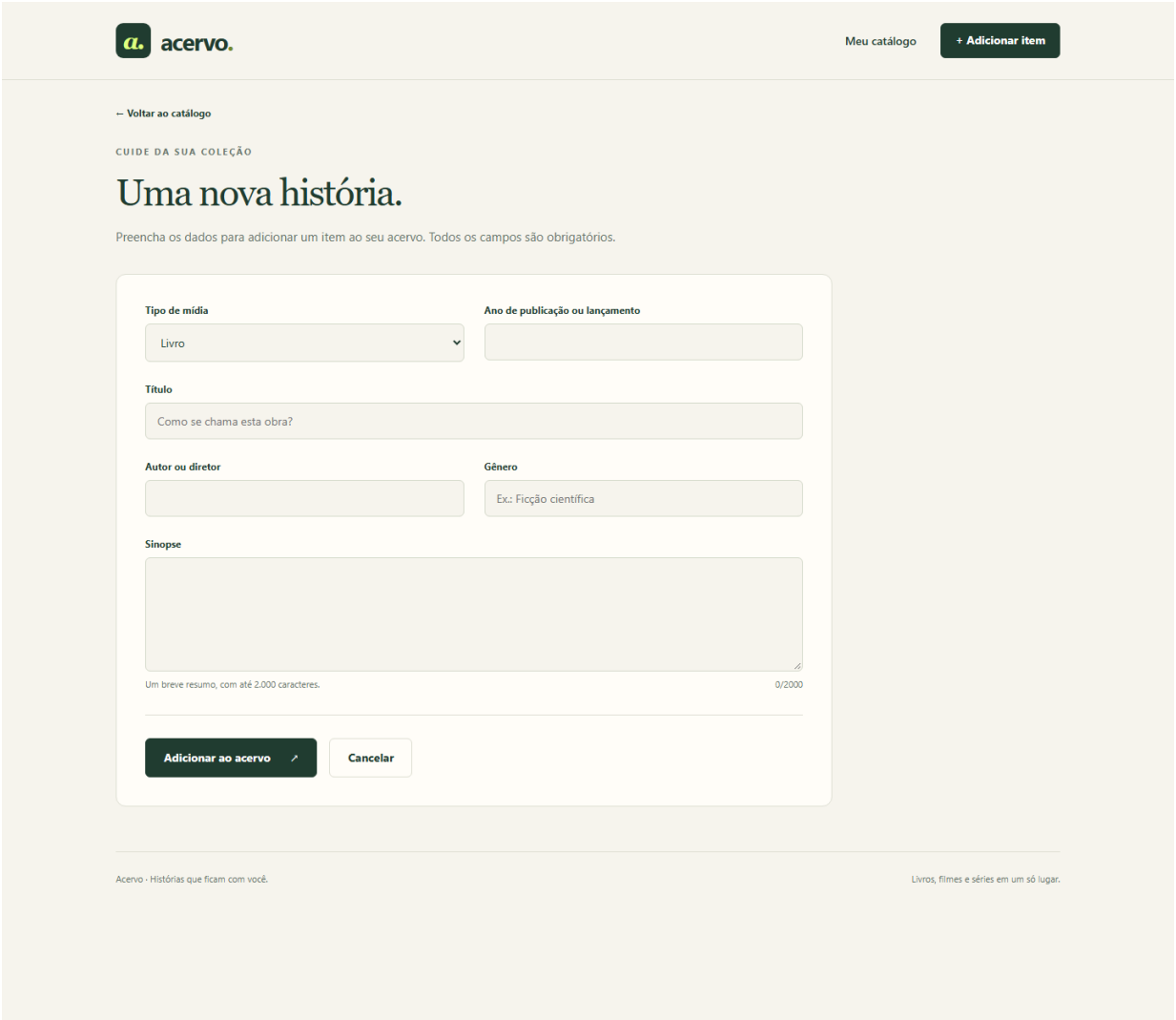
O registro factual de testes, contagens, falhas e ambiente acompanha o arquivo VERIFICACAO.md. Não foram realizados ensaios de carga, auditoria de segurança externa ou validação com usuários finais. As capturas de tela registram uma execução de demonstração com dados fictícios de uso do catálogo e resumos originais.

10. Resultados visuais



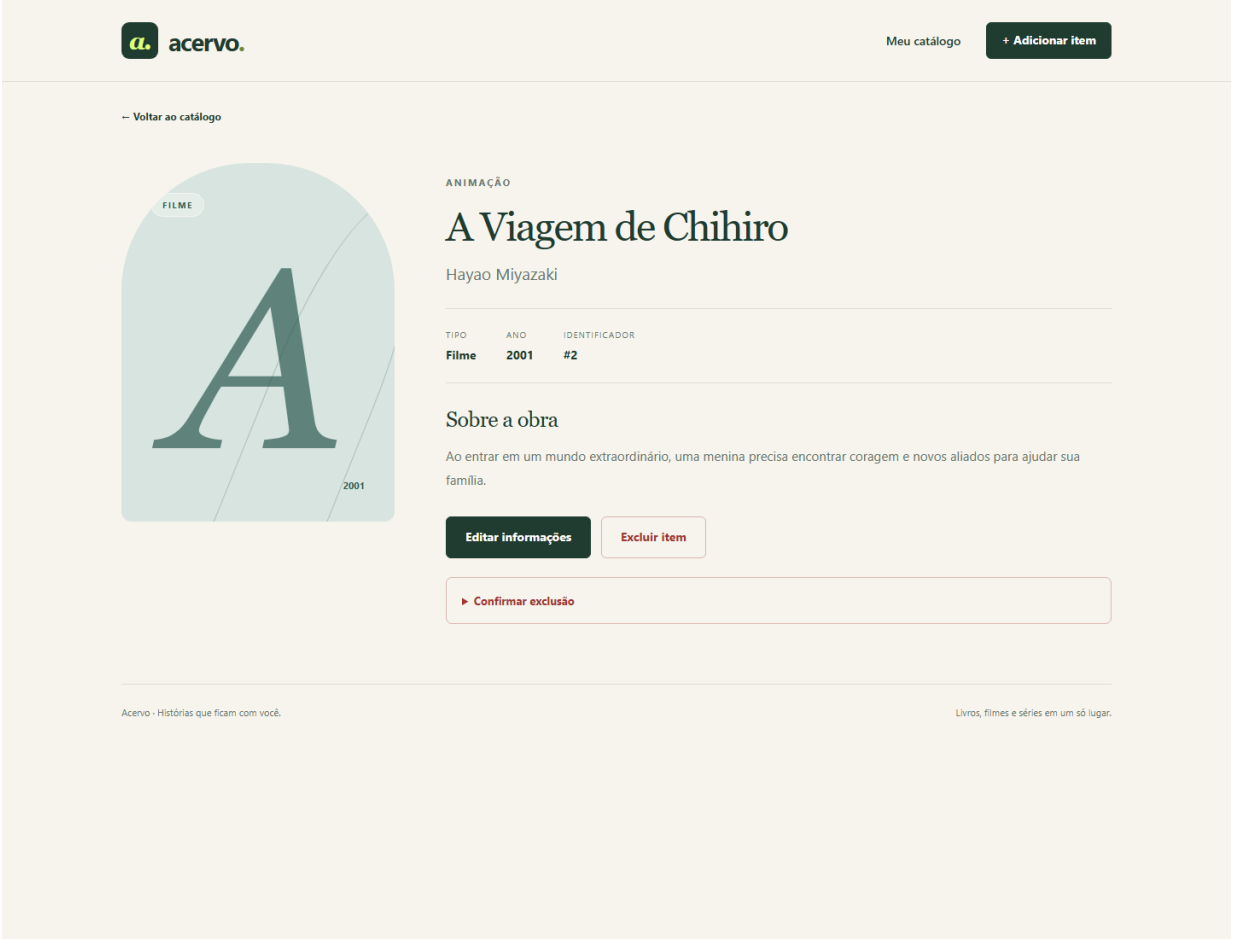
Listagem do catálogo em execução

Figura 1. Coleção demonstrativa com seis obras. A interface apresenta tipo, título, autor/diretor, ano e acesso aos detalhes. As artes são tipográficas e geradas por CSS.



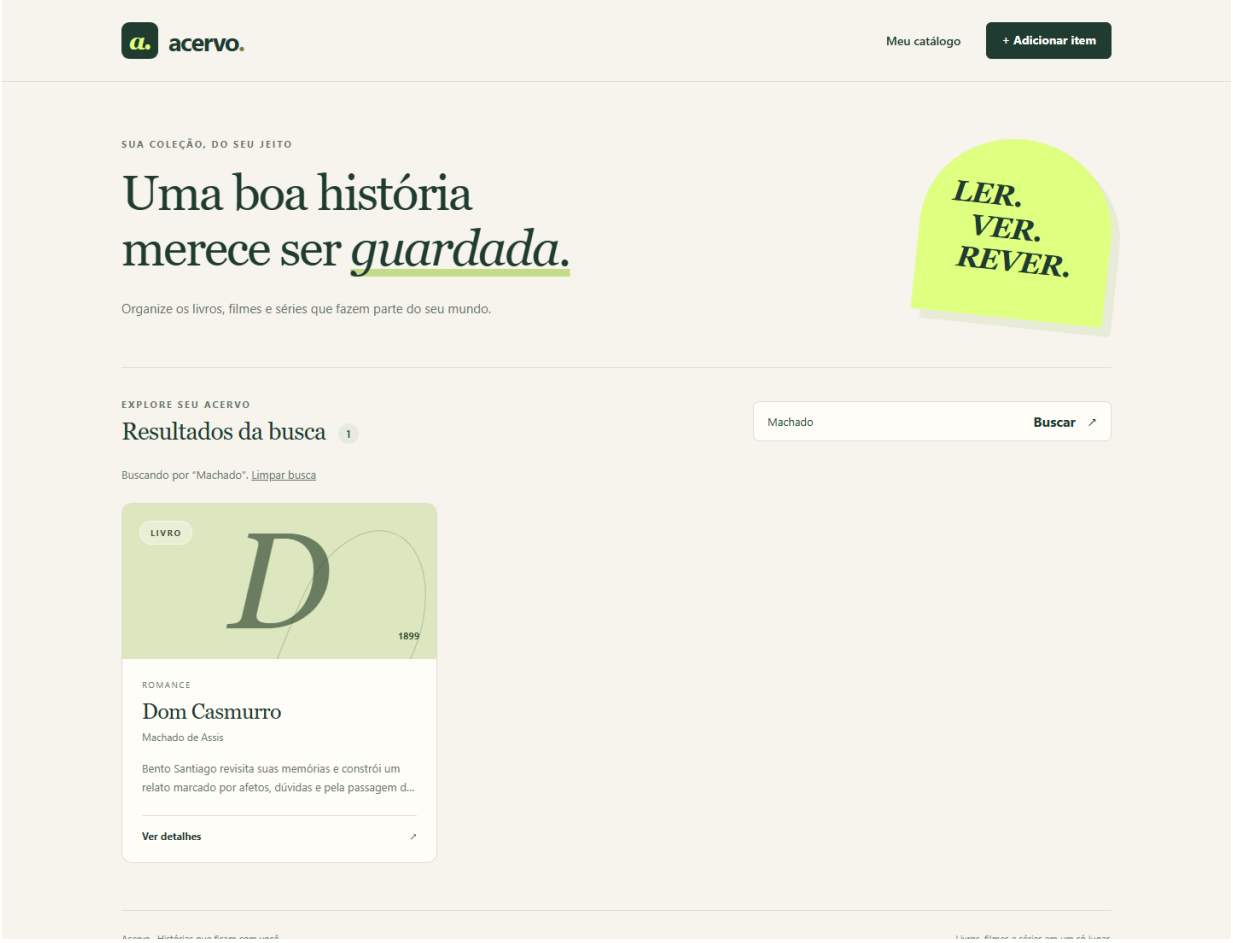
Formulário de cadastro

Figura 2. Formulário com seleção de mídia, campos obrigatórios e contador de sinopse.



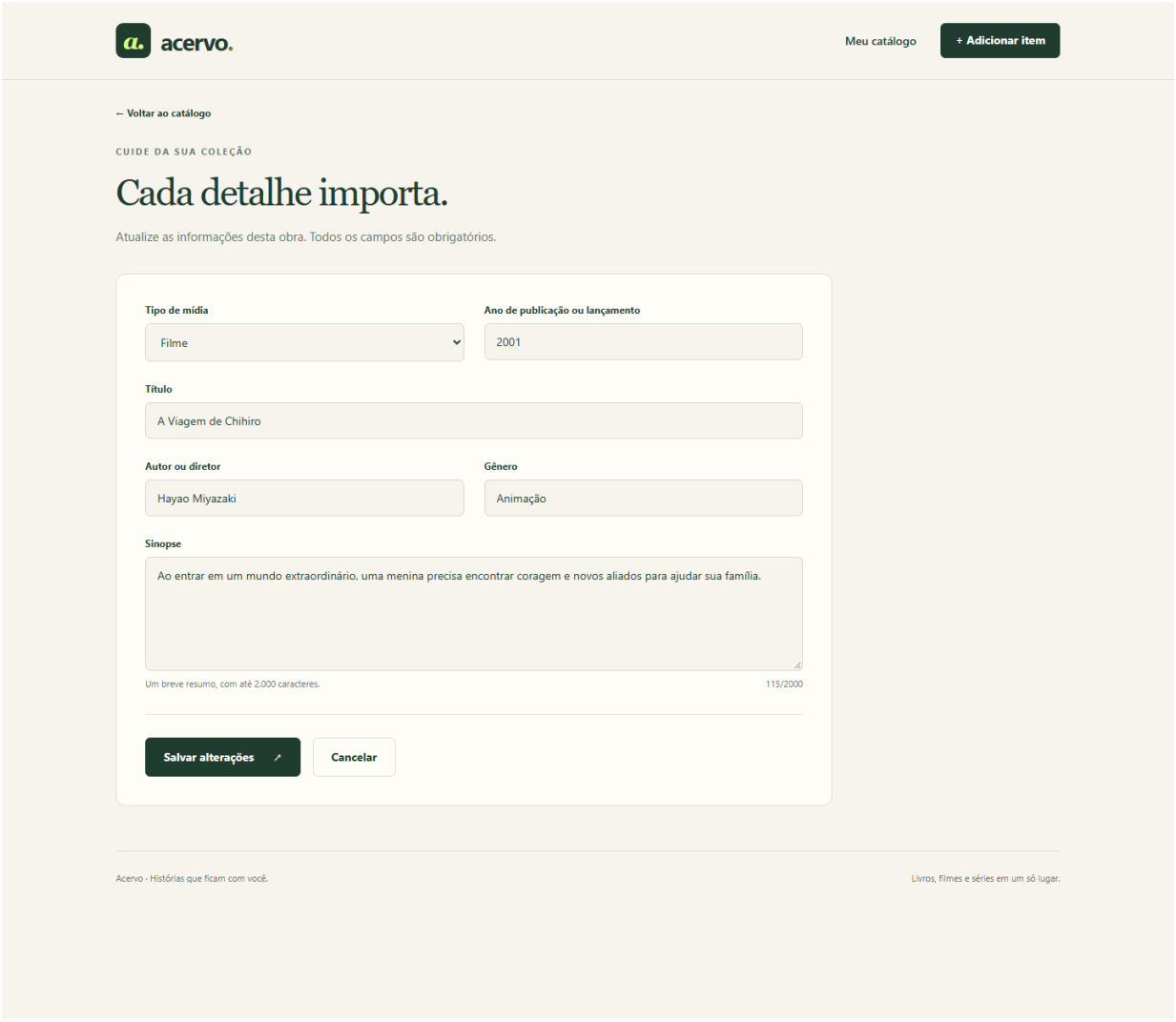
Detalhes de uma obra

Figura 3. Informações completas, acesso à edição e confirmação de exclusão.



Busca em execução

Figura 4. Busca por autor no catálogo demonstrativo.



Formulário de edição

Figura 5. Reaproveitamento do formulário com dados existentes e identificação do item.



Catálogo em tela pequena

Figura 6. Layout de listagem em uma viewport de celular, registrado sem alteração do conteúdo funcional.

11. Discussão e respostas à situação-problema

Como a POO melhora a estrutura? Ela permite representar a obra como entidade e atribuir comportamentos a classes com responsabilidades claras. O controlador não precisa saber como o banco armazena o item e o serviço não depende de HTTP. O efeito prático é localizar uma mudança no componente correspondente à sua natureza.

Qual é o papel do DAO? Ele encapsula o armazenamento e o CRUD. A implementação entregue reutiliza o mapeamento do ResultSet, o preenchimento dos parâmetros e a estrutura de consultas e modificações. O contrato torna possível substituir a implementação sem alterar a lógica de validação do serviço. A migração para outro SGBD ainda exige adaptar esquema, driver e particularidades SQL; a interface não torna bancos automaticamente intercambiáveis.

Como o serviço melhora a organização e testabilidade? Ele concentra as regras obrigatórias antes da escrita e coordena as operações. Seus testes instanciam um serviço com DAO em memória e relógio fixo, demonstrando que as regras são verificáveis sem servidor ou banco. Isso reduz o acoplamento dos testes de negócio à infraestrutura.

Os desafios técnicos incluem manter a compatibilidade de pacotes jakarta entre Servlet, JSP e Tomcat; conferir a descoberta das bibliotecas de tags; preservar os valores do formulário após falhas; tratar recursos e exceções na camada correta; e evitar regras repetidas entre cadastro e edição. Os testes de integração verificam exatamente essas fronteiras que testes isolados não alcançam.

12. Conclusão e trabalhos futuros

O projeto fornece um catálogo completo, uma arquitetura em camadas, testes automatizados e documentação explicativa. A solução atende à Situação-Problema 3 ao retirar persistência e regras da Servlet e ao demonstrar reutilização e substituição de dependências. A manutenção melhora pela distribuição das responsabilidades, sem atribuir um ganho quantitativo não medido.

Como evolução, propõem-se paginação, filtros de gênero, autenticação, auditoria de alterações, controle de concorrência por versão e migração planejada para um SGBD servidor. A entrega mantém um escopo compatível com o projeto acadêmico e com a compreensão detalhada pelo estudante.

13. Referências utilizadas

MARQUES, Artur. Projetos Computacionais: da Teoria à Prática. Material teórico da disciplina, arquivo teorico (1).pdf fornecido pelo estudante, 44 páginas. Sem data editorial identificada. Base do problema: pp. 39–40; requisitos e documentação: pp. 7–9 e 20–28.

APACHE SOFTWARE FOUNDATION. Apache Tomcat 10.1 Documentation. Disponível em: https://tomcat.apache.org/tomcat-10.1-doc/index.html. Consultado em 17 out. 2026.

APACHE SOFTWARE FOUNDATION. Apache Maven: Download and Documentation. Disponível em: https://maven.apache.org/download.cgi. Consultado em 17 out. 2026. A entrega fixa Maven 3.9.9 como ferramenta local, sem afirmar que seja a versão mais recente.

ECLIPSE FOUNDATION. Jakarta Standard Tag Library 3.0. Disponível em: https://jakarta.ee/specifications/tags/3.0/. Consultado em 17 out. 2026.

H2 DATABASE ENGINE. Features. Disponível em: https://h2database.com/html/features.html. Consultado em 17 out. 2026.

JUNIT TEAM. JUnit 5.12.2 Release Notes. Disponível em: https://docs.junit.org/5.12.2/release-notes/. Consultado em 17 out. 2026.

As obras bibliográficas mencionadas pelo material da disciplina não são apresentadas aqui como leituras diretas realizadas nesta implementação. A revisão conceitual se apoia no próprio PDF fornecido e nas documentações oficiais efetivamente consultadas.